Binary Search Trees (BST)

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University



Outline

1. Introduction
2. Applications of BST
3. BST Example
4. Searching in BST
5. Insertion in BST
6. Deletion in BST

What is a BST?

- A **binary search tree (BST)** is a binary tree with an extra **ordering property**.
- Each node stores a **key** (and optionally, value/record).
- **BST property** for every node u:
  - All keys in the **left** subtree of u are $\leq$ key of u.
  - All keys in the **right** subtree of u are $\geq$ key of u.
- This structure lets us **search**, **insert**, and **delete** in time proportional to the tree height.

BST Node Definition (C++)

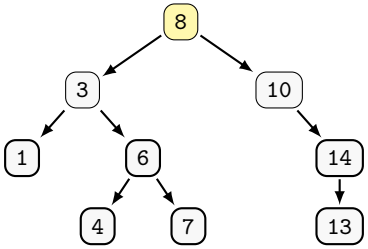
```cpp
struct Node {
  int key;
  Node* left;
  Node* right;
  Node(int k): key(k), left(nullptr), right(nullptr) {}
};
```

Applications of BST

- Symbol tables / dictionaries
- Maintaining a dynamic set of keys
- Search in $\mathcal{O}(h)$
- Insert / delete in $\mathcal{O}(h)$
- Implementing `map`/`set`-like containers

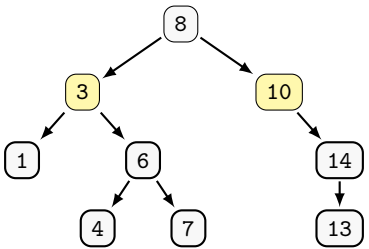
- Find **min/max**, **successor/predecessor**
- Range queries (keys in [L, R])
- Basis for self-balancing trees (AVL, Red-Black)

Example BST (Valid)



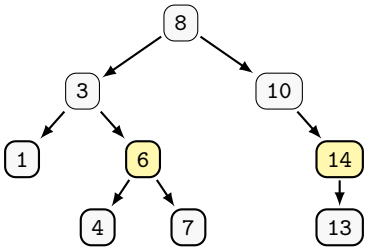
- Left of 8: {3,1,6,4,7} all ≤ 8
- Right of 8: {10,14,13} all ≥ 8
- Recursively true for each subtree $\Rightarrow$ **Valid BST**

Example BST (Valid)



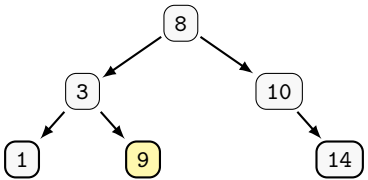
- Left of 3: {1} all ≤ 3
- Right of 3: {4,6,7} all ≥ 3
- Left of 10: N/A
- Right of 10: {13,14} all ≥ 10

Example BST (Valid)



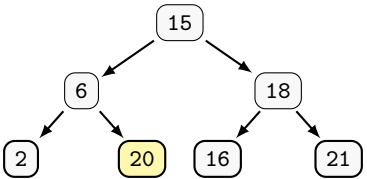
- Left of 6: {4} all ≤ 6
- Right of 6: {7} all ≥ 6
- Left of 14: {13} all ≤ 14
- Right of 14: N/A

Example: Invalid BST



- Node 9 is in the **left** subtree of 8, but $9 > 8$.
- So the **global BST property** is broken.

Example: Invalid BST



- Node 20 is in the **left** subtree of 15, but $20 > 15$.
- Local parent (6) is okay, but higher ancestor (15) is violated.

Operations in a BST

Common operations on a Binary Search Tree (BST):

- **Searching** for a key.
- **Insertion** of a new key.
- **Deletion** of a key.
- Traversal (**Inorder**, **Preorder**, **Postorder**).
- Finding **min**, **max**, **successor**, **predecessor**.

All these operations rely on the BST property for correctness and efficiency.

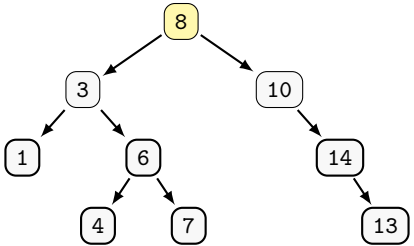
Searching in BST (C++ Code)

```
Node* searchBST(Node* root, int key){
  if(!root) return nullptr;
  if(root->key == key) return root;
  if(key < root->key)
    return searchBST(root->left, key);
  else
    return searchBST(root->right, key);
}
```

Idea: recursively narrow the search to left or right subtree based on key comparison.

BST Search Simulation (search key = 7)

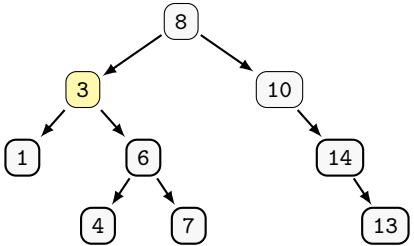
Goal: Find key 7 in the BST.



Compare 7 with **8** $\rightarrow$ $7 < 8$ $\rightarrow$ go **left**.

BST Search Simulation (search key = 7)

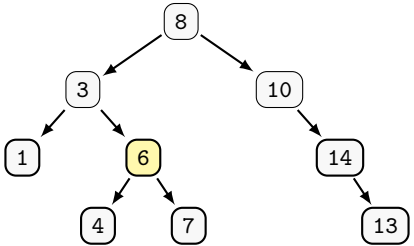
Goal: Find key 7 in the BST.



Compare 7 with **3** $\rightarrow$ $7 > 3$ $\rightarrow$ go **right**.

BST Search Simulation (search key = 7)

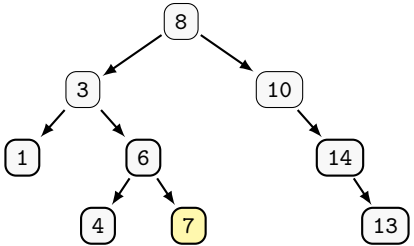
Goal: Find key 7 in the BST.



Compare 7 with **6** $\rightarrow$ $7 > 6$ $\rightarrow$ go **right**.

BST Search Simulation (search key = 7)

Goal: Find key 7 in the BST.



Compare with **7** $\rightarrow$ **found**.

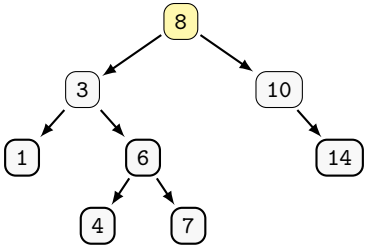
Insertion in BST (C++ Code)

```
Node* insertBST(Node* root, int key){
  if(!root) return new Node(key);
  if(key < root->key)
    root->left = insertBST(root->left, key);
  else if(key > root->key)
    root->right = insertBST(root->right, key);
  return root;
}
```

Idea: recursively find correct position (null link) for the new key while maintaining BST property.

BST Insertion Simulation (insert key = 5)

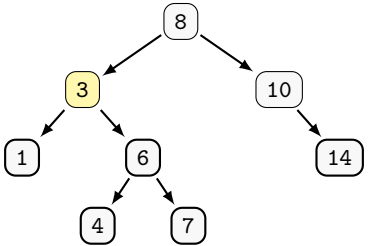
Goal: Insert key 5 into the BST.



$5 < 8 \rightarrow$ go **left**.

BST Insertion Simulation (insert key = 5)

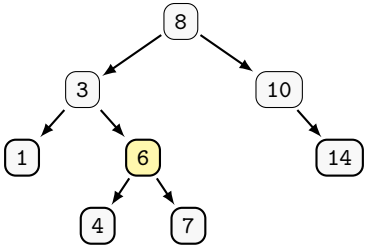
Goal: Insert key 5 into the BST.



$5 > 3 \rightarrow$ go **right**.

BST Insertion Simulation (insert key = 5)

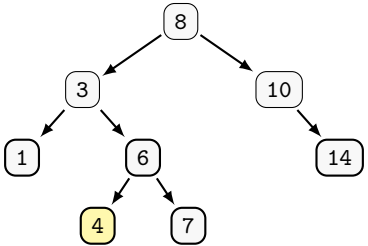
Goal: Insert key 5 into the BST.



$5 < 6 \rightarrow$ go **left**.

BST Insertion Simulation (insert key = 5)

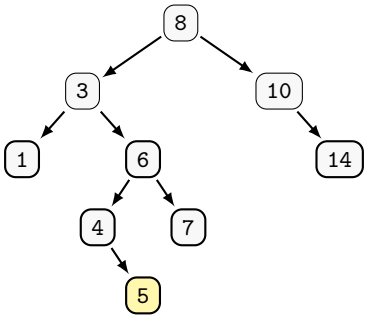
Goal: Insert key 5 into the BST.



$5 > 4 \rightarrow$ go **right**.

BST Insertion Simulation (insert key = 5)

Goal: Insert key 5 into the BST.



Empty spot found $\rightarrow$ insert **5** here.

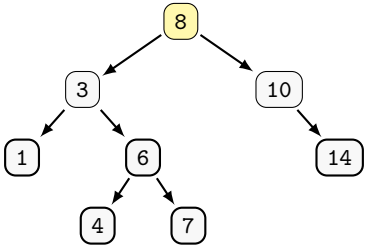
Deletion in BST (C++ Code)

```
Node* findMin(Node* root){
  while(root->left) root = root->left;
  return root;
}

Node* deleteBST(Node* root, int key){
  if(!root) return nullptr;
  if(key < root->key)
    root->left = deleteBST(root->left, key);
  else if(key > root->key)
    root->right = deleteBST(root->right, key);
  else {
    if(!root->left) return root->right;
    else if(!root->right) return root->left;
    Node* succ = findMin(root->right);
    root->key = succ->key;
    root->right = deleteBST(root->right, succ->key);
  }
  return root;
}
```

BST Deletion Simulation (delete key = 3)

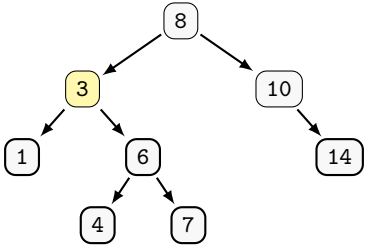
Goal: Delete key 3 (node with two children).



$3 < 8 \rightarrow$ go **left**.

BST Deletion Simulation (delete key = 3)

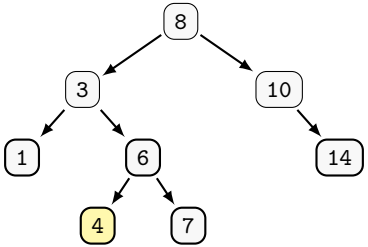
Goal: Delete key 3 (node with two children).



Node **3** has two children $\rightarrow$ need inorder successor (min in right subtree).

BST Deletion Simulation (delete key = 3)

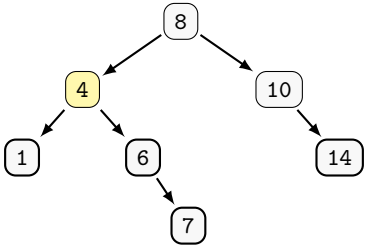
Goal: Delete key 3 (node with two children).



Inorder successor of 3 is **4**. Copy 4 into node 3.

BST Deletion Simulation (delete key = 3)

Goal: Delete key 3 (node with two children).



Final shape after deleting original 4 from the right subtree.